

Lesson 3 Notes: Complete C Sparse Matrix Display Program

1. Where we are now

Before Lesson 3, we already learned two important ideas.

Lesson 1 idea

A full matrix does not always need full storage.

For example, in a diagonal matrix, useful values are only found where:

```
i == j
```

All other positions are structural zeros.

So instead of storing the full matrix, we store only the useful diagonal values.

Lesson 2 idea

A sparse matrix is different from a diagonal matrix.

In a sparse matrix, non-zero values can appear anywhere.

So each non-zero value must store three things:

```
row, column, value
```

This is called Coordinate List representation.

Example:

```
(0, 3, 7)
```

means:

```
At row 0, column 3, the value is 7.
```

By the end of Lesson 2, we knew how to create a sparse matrix representation.

Lesson 3 teaches how to display that sparse matrix as a full matrix.

2. Main idea of Lesson 3

In a sparse matrix, we store only non-zero elements.

But when we display the matrix, we usually want to see the full grid.

Example full matrix:

```
1 0 0 0 0
1 0 0 0 0
1 0 0 0 0
1 0 0 0 0
1 0 0 0 0
```

This matrix has 25 positions because it is a `5 x 5` matrix.

But only 5 values are non-zero.

So we store only these 5 records:

```
(0, 0, 1)
(1, 0, 1)
(2, 0, 1)
(3, 0, 1)
(4, 0, 1)
```

The main question in Lesson 3 is:

How can we print the full matrix even though we stored only the non-zero values?

The answer is:

1. Visit every position in the full matrix.
 2. Check whether the current position is stored as a non-zero element.
 3. If it is stored, print the stored value.
 4. If it is not stored, print `0`.
-

3. Important indexing idea

In beginner explanations, matrix positions are sometimes written using 1-based indexing.

That means row and column numbers start from 1.

Example:

```
(1, 1)  
(1, 2)  
(2, 1)
```

But C arrays and C loops usually use 0-based indexing.

That means row and column numbers start from 0.

Example:

```
(0, 0)  
(0, 1)  
(1, 0)
```

In Lesson 3, the C display program uses loops like this:

```
for (i = 0; i < s.m; i++)
```

This means the program starts from row **0**, not row **1**.

So for a **5 x 5** matrix, valid row indexes are:

```
0, 1, 2, 3, 4
```

Valid column indexes are also:

```
0, 1, 2, 3, 4
```

So for Lesson 3, we enter positions like this:

```
0 0 1
1 0 1
2 0 1
3 0 1
4 0 1
```

Not like this:

```
1 1 1
2 1 1
3 1 1
4 1 1
5 1 1
```

Why this matters

If the program uses 0-based loops but the user enters 1-based positions, the displayed matrix will be wrong.

So we must be consistent.

In this lesson:

Use 0-based row and column indexes.

4. Structures used in the program

The program uses two structures:

1. `Element`
2. `Sparse`

4.1 The `Element` structure

One `Element` stores one non-zero value.

```
struct Element {
    int i; // row index
```

```
int j; // column index
int x; // value
};
```

Meaning of each field

Field	Meaning
i	row index
j	column index
x	value stored at that row and column

Example:

```
struct Element e;
e.i = 2;
e.j = 4;
e.x = 9;
```

This means:

At row 2, column 4, the value is 9.

So one `Element` does not store the whole matrix.

It stores only one non-zero value.

4.2 The `Sparse` structure

The `Sparse` structure stores information about the whole sparse matrix.

```
struct Sparse {
    int m;           // number of rows
    int n;           // number of columns
    int num;         // number of non-zero elements
    struct Element *ele; // dynamic array of non-zero elements
};
```

Meaning of each field

Field	Meaning
<code>m</code>	total number of rows
<code>n</code>	total number of columns
<code>num</code>	number of non-zero elements
<code>ele</code>	pointer to the dynamic array of stored elements

Example:

```
m = 5  
n = 5  
num = 5
```

This means:

```
The matrix has 5 rows.  
The matrix has 5 columns.  
The matrix has 5 non-zero values.
```

Important:

```
num does not mean m * n.
```

For a `5 x 5` matrix:

```
m * n = 25
```

But if only 5 values are non-zero:

```
num = 5
```

5. Why we use `malloc`

In C, we use `malloc` to create memory dynamically.

Dynamic memory means memory created while the program is running.

We use dynamic memory because we do not know how many non-zero elements the user will enter until the program runs.

The user might enter:

```
num = 3
```

or:

```
num = 50
```

or:

```
num = 1000
```

So we first read `num`, then create exactly enough space.

Correct allocation:

```
s->ele = (struct Element *) malloc(s->num * sizeof(struct Element));
```

This means:

Create space for num Element records.

If:

```
num = 5
```

then this creates space for:

```
ele[0]  
ele[1]  
ele[2]  
ele[3]  
ele[4]
```

We do not create space for all `m * n` positions.

That would defeat the purpose of sparse matrix storage.

6. Header files used

The complete C program uses these header files:

```
#include <stdio.h>
#include <stdlib.h>
```

`stdio.h`

We include `stdio.h` because we use:

```
printf()
scanf()
```

`printf()` prints output.

`scanf()` reads input.

`stdlib.h`

We include `stdlib.h` because we use:

```
malloc()
free()
exit()
```

`malloc()` creates dynamic memory.

`free()` releases dynamic memory.

`exit()` stops the program if something serious goes wrong, such as memory allocation failure.

7. The `create()` function

The `create()` function builds the sparse matrix representation.

It reads:

1. Matrix dimensions
2. Number of non-zero elements
3. Each non-zero element

7.1 Code for `create()`

```
void create(struct Sparse *s) {
    int i;

    printf("Enter matrix dimensions: ");
    scanf("%d %d", &s->m, &s->n);

    printf("Enter number of non-zero elements: ");
    scanf("%d", &s->num);

    s->ele = (struct Element *) malloc(s->num * sizeof(struct Element));

    if (s->ele == NULL) {
        printf("Memory allocation failed.\n");
        exit(1);
    }

    printf("Enter row, column, and value for each non-zero element:\n");
    for (i = 0; i < s->num; i++) {
        scanf("%d %d %d", &s->ele[i].i, &s->ele[i].j, &s->ele[i].x);
    }
}
```

7.2 Why `create()` receives a pointer

The function header is:

```
void create(struct Sparse *s)
```

Here, `s` is a pointer to a `Sparse` structure.

We pass a pointer because `create()` must modify the original sparse matrix in `main()`.

In `main()`, we call:

```
create(&s);
```

The symbol `&` means address.

So:

```
&s
```

means:

```
address of s
```

Because `create()` receives the address of `s`, it can change the original structure.

It can fill:

```
s.m  
s.n  
s.num  
s.ele
```

7.3 Why we use `->` inside `create()`

Inside `create()`, `s` is a pointer.

When we access a structure through a pointer, we use the arrow operator:

```
->
```

Correct:

```
s->m  
s->n  
s->num  
s->ele
```

Wrong inside `create()` :

```
s.m  
s.n  
s.num  
s.ele
```

Simple rule:

If you have a pointer, use `->`.
If you have the actual structure variable, use `.`

8. The display problem

After `create()` runs, the sparse matrix might be stored like this:

```
m = 5  
n = 5  
num = 5  
  
ele[0] = {0, 0, 1}  
ele[1] = {1, 0, 1}  
ele[2] = {2, 0, 1}  
ele[3] = {3, 0, 1}  
ele[4] = {4, 0, 1}
```

This is compact storage.

But when we display the matrix, we want this full output:

```
1 0 0 0 0  
1 0 0 0 0  
1 0 0 0 0
```

```
1 0 0 0 0
1 0 0 0 0
```

The important point is:

The full matrix is not stored.

The program reconstructs the full matrix while printing.

9. The tracker variable `k`

The display function uses a variable called `k`.

```
int k = 0;
```

The variable `k` tracks the current non-zero element inside the `ele` array.

At the beginning:

```
k = 0
```

So the current stored element is:

```
ele[0]
```

After printing that non-zero value, we do:

```
k++;
```

Now `k` moves to the next stored element.

So:

```
k = 1
```

means the current stored element is:

```
ele[1]
```

Beginner mental model for `k`

Think of `k` as a finger pointing at the next non-zero value.

The display function moves across the full matrix cell by cell.

The variable `k` points to the next stored non-zero element that might need to be printed.

10. Display logic

The display function uses nested loops.

The outer loop moves row by row.

The inner loop moves column by column.

```
for (i = 0; i < s.m; i++) {  
    for (j = 0; j < s.n; j++) {  
        // check current position  
    }  
}
```

For each position `(i, j)`, the program checks:

```
if (k < s.num && i == s.ele[k].i && j == s.ele[k].j)
```

This condition asks:

1. Is `k` still inside the stored element array?
2. Is the current row equal to the row of `ele[k]`?
3. Is the current column equal to the column of `ele[k]`?

If all are true, the current matrix position contains a stored non-zero value.

So we print:

```
printf("%d ", s.ele[k].x);
```

Then we move to the next stored element:

```
k++;
```

If the condition is false, the current position is not stored.

So it must be zero.

We print:

```
printf("0 ");
```

11. The `display()` function

```
void display(struct Sparse s) {
    int i, j, k = 0;

    for (i = 0; i < s.m; i++) {
        for (j = 0; j < s.n; j++) {
            if (k < s.num && i == s.ele[k].i && j == s.ele[k].j) {
                printf("%d ", s.ele[k].x);
                k++;
            } else {
                printf("0 ");
            }
        }
        printf("\n");
    }
}
```

11.1 Why `display()` receives the structure by value

The function header is:

```
void display(struct Sparse s)
```

Here, `s` is not a pointer.

It is a normal structure variable inside the function.

The display function only needs to read the matrix and print it.

It does not need to change the original matrix.

So passing by value is enough for this lesson.

11.2 Why we use `.` inside `display()`

Inside `display()`, `s` is a normal structure variable.

So we use the dot operator:

```
.
```

Correct:

```
s.m  
s.n  
s.num  
s.ele
```

Wrong inside `display()` :

```
s->m  
s->n  
s->num  
s->ele
```

Simple rule again:

```
Pointer: use ->  
Object/value: use .
```

12. Complete C program

```
#include <stdio.h>
#include <stdlib.h>

struct Element {
    int i; // row index
    int j; // column index
    int x; // value
};

struct Sparse {
    int m;           // number of rows
    int n;           // number of columns
    int num;         // number of non-zero elements
    struct Element *ele; // dynamic array
};

void create(struct Sparse *s) {
    int i;

    printf("Enter matrix dimensions: ");
    scanf("%d %d", &s->m, &s->n);

    printf("Enter number of non-zero elements: ");
    scanf("%d", &s->num);

    s->ele = (struct Element *) malloc(s->num * sizeof(struct Element));

    if (s->ele == NULL) {
        printf("Memory allocation failed.\n");
        exit(1);
    }

    printf("Enter row, column, and value for each non-zero element:\n");
    for (i = 0; i < s->num; i++) {
        scanf("%d %d %d", &s->ele[i].i, &s->ele[i].j, &s->ele[i].x);
    }
}

void display(struct Sparse s) {
    int i, j, k = 0;

    for (i = 0; i < s.m; i++) {
        for (j = 0; j < s.n; j++) {
            if (k < s.num && i == s.ele[k].i && j == s.ele[k].j) {
```



```

        printf("%d ", s.ele[k].x);
        k++;
    } else {
        printf("0 ");
    }
}
printf("\n");
}
}

int main() {
    struct Sparse s;

    create(&s);

    printf("\nSparse Matrix Display:\n");
    display(s);

    free(s.ele);

    return 0;
}

```

13. Explanation of `main()`

```

int main() {
    struct Sparse s;

```

This creates a sparse matrix structure variable named `s`.

At this point, its fields do not have useful values yet.

```

create(&s);

```

This sends the address of `s` to `create()`.

The `create()` function fills the matrix information.

```
printf("\nSparse Matrix Display:\n");  
display(s);
```

This prints a heading and then displays the matrix as a full grid.

```
free(s.ele);
```

This releases the dynamic memory created by `malloc`.

This is important because memory created with `malloc` should be released using `free` when it is no longer needed.

```
return 0;
```

This ends the program successfully.

14. Example input

```
Enter matrix dimensions: 5 5  
Enter number of non-zero elements: 5  
Enter row, column, and value for each non-zero element:  
0 0 1  
1 0 1  
2 0 1  
3 0 1  
4 0 1
```

This means:

```
The matrix has 5 rows.  
The matrix has 5 columns.  
There are 5 non-zero values.
```

The stored values are:

```
row 0, column 0, value 1
row 1, column 0, value 1
row 2, column 0, value 1
row 3, column 0, value 1
row 4, column 0, value 1
```

15. Expected output

```
Sparse Matrix Display:
1 0 0 0 0
1 0 0 0 0
1 0 0 0 0
1 0 0 0 0
1 0 0 0 0
```

This output has a first column of `1` s.

All other values are `0` .

16. Dry run of `display()`

Stored data:

```
ele[0] = {0, 0, 1}
ele[1] = {1, 0, 1}
ele[2] = {2, 0, 1}
ele[3] = {3, 0, 1}
ele[4] = {4, 0, 1}
```

Initial value:

```
k = 0
```

16.1 First row

The first row has:

```
i = 0
```

Position (0, 0)

Current position:

```
i = 0  
j = 0
```

Current stored element:

```
ele[k] = ele[0] = {0, 0, 1}
```

Check:

```
i == ele[k].i -> 0 == 0 -> true  
j == ele[k].j -> 0 == 0 -> true
```

So the position matches.

Print:

```
1
```

Then move to the next stored element:

```
k = 1
```

Position (0, 1)

Current position:

```
i = 0  
j = 1
```

Current stored element:

```
ele[1] = {1, 0, 1}
```

Check:

```
i == ele[1].i -> 0 == 1 -> false
```

So there is no stored value at `(0, 1)`.

Print:

```
0
```

The same thing happens for:

```
(0, 2)  
(0, 3)  
(0, 4)
```

So the first row is:

```
1 0 0 0 0
```

16.2 Second row

Now:

```
i = 1
```

Position `(1, 0)`

Current position:

```
i = 1  
j = 0
```

Current stored element:

```
ele[1] = {1, 0, 1}
```

Check:

```
i == ele[1].i -> 1 == 1 -> true  
j == ele[1].j -> 0 == 0 -> true
```

So the position matches.

Print:

```
1
```

Then:

```
k = 2
```

The rest of the row has no stored values.

So the second row is:

```
1 0 0 0 0
```

16.3 Remaining rows

The same logic continues.

At `(2, 0)`, the program prints `1`.

At `(3, 0)`, the program prints `1`.

At `(4, 0)`, the program prints `1`.

All other positions are printed as `0`.

Final output:

```
1 0 0 0 0
1 0 0 0 0
1 0 0 0 0
1 0 0 0 0
1 0 0 0 0
```

17. Important assumption: row-major order

The display algorithm assumes the stored elements are entered in row-major order.

Row-major order means:

1. Sort by row first.
2. If two elements are in the same row, sort by column.

Good order:

```
0 0 1
1 0 1
2 0 1
3 0 1
4 0 1
```

This order moves from top to bottom.

Another good example:

```
0 1 5
0 3 9
2 0 4
3 2 8
```

Here, row **0** entries come first.

Inside row **0**, column **1** comes before column **3**.

Then later rows come after.

Bad order:

```
4 0 1
0 0 1
2 0 1
1 0 1
3 0 1
```

This is bad because the display function uses `k` moving forward only.

It does not search the whole `ele` array again and again.

So if the elements are not stored in order, the display output can become wrong.

18. Why the condition uses `k < s.num`

The display condition is:

```
if (k < s.num && i == s.ele[k].i && j == s.ele[k].j)
```

The first part is:

```
k < s.num
```

This checks whether `k` is still inside the valid range of the element array.

Example:

If:

```
num = 5
```

then valid indexes are:

```
0, 1, 2, 3, 4
```

When `k` becomes `5`, there is no valid `ele[5]`.

So this is unsafe:


```
s.ele[k]
```

when:

```
k == s.num
```

The condition `k < s.num` prevents the program from reading outside the array.

This is a safer version of the display code.

19. Complexity analysis

19.1 Complexity of `create()`

The `create()` function reads each non-zero element once.

If there are `num` non-zero elements, the loop runs `num` times.

So:

```
create() time complexity = O(num)
```

The function also creates an array of `num` elements.

So:

```
create() space complexity = O(num)
```

19.2 Complexity of `display()`

The `display()` function visits every position in the full matrix.

For a matrix with `m` rows and `n` columns, the total number of positions is:

```
m * n
```

So:

```
display() time complexity =  $O(m * n)$ 
```

The display function only uses a few variables:

```
i, j, k
```

So it does not need extra memory that grows with the matrix size.

Therefore:

```
display() extra space complexity =  $O(1)$ 
```

19.3 Important complexity idea

Sparse storage saves memory.

But displaying the full matrix still takes full matrix time.

Why?

Because display must print every visible position.

Even if most values are zero, the screen output still includes all positions.

So:

```
Storage can be  $O(\text{num})$ , but display is  $O(m * n)$ .
```

20. Complexity table

Operation	Time Complexity	Space Complexity	Reason
Create sparse matrix	$O(\text{num})$	$O(\text{num})$	Reads and stores each non-zero element once

Operation	Time Complexity	Space Complexity	Reason
Display sparse matrix	$O(m * n)$	$O(1)$ extra	Visits every visible matrix position
Store full normal matrix	—	$O(m * n)$	Stores every row-column position
Store sparse matrix	—	$O(\text{num})$	Stores only non-zero elements

21. Common beginner mistakes

Mistake 1: Using `.` inside `create()`

Wrong:

```
s.m
```

Correct:

```
s->m
```

Reason:

Inside `create()`, `s` is a pointer.

Pointer fields are accessed using `->`.

Mistake 2: Using `->` inside `display()`

Wrong:

```
s->m
```

Correct:

```
s.m
```

Reason:

Inside `display()`, `s` is a normal structure variable, not a pointer.

Normal structure fields are accessed using `.`.

Mistake 3: Allocating `m * n` elements

Wrong:

```
s->ele = (struct Element *) malloc(s->m * s->n * sizeof(struct Element));
```

This stores space for every matrix position.

That wastes memory.

Correct:

```
s->ele = (struct Element *) malloc(s->num * sizeof(struct Element));
```

This stores space only for non-zero elements.

Mistake 4: Forgetting `k < s.num`

Unsafe:

```
if (i == s.ele[k].i && j == s.ele[k].j)
```

Safer:

```
if (k < s.num && i == s.ele[k].i && j == s.ele[k].j)
```

Reason:

The safer version avoids reading outside the array after all non-zero elements have been printed.

Mistake 5: Entering elements in random order

Wrong order:

```
4 0 1
0 0 1
2 0 1
1 0 1
3 0 1
```

Correct row-major order:

```
0 0 1
1 0 1
2 0 1
3 0 1
4 0 1
```

Reason:

The display algorithm assumes the stored elements are sorted by row and column.

Mistake 6: Mixing 1-based and 0-based indexing

If the display loop starts from `0`, enter positions starting from `0`.

For a `5 x 5` matrix, use:

```
0 to 4
```

not:

```
1 to 5
```

Mistake 7: Forgetting to free memory

If memory is created using `malloc`, it should be released using `free`.

Correct:

```
free(s. ele);
```

This releases the dynamic array after the program finishes using it.

22. Beginner mental model

Think of the full matrix as a large grid.

Most positions are empty.

The sparse matrix stores only the positions that contain real values.

Example stored list:

```
(0, 0, 1)
(1, 0, 1)
(2, 0, 1)
(3, 0, 1)
(4, 0, 1)
```

This means:

```
There is a 1 in the first column of every row.
```

When displaying, the program walks through the full grid.

For each cell, it asks:

```
Is this cell in the stored list?
```

If yes, print the stored value.

If no, print zero.

So the full matrix is recreated only when needed.

23. One-line memory trick

A sparse matrix stores only the non-zero values, but `display()` prints the full grid by filling missing positions with zero.

24. Important code symbols

Symbol	Meaning
<code>struct Element</code>	Structure for one non-zero value
<code>struct Sparse</code>	Structure for the whole sparse matrix
<code>m</code>	Number of rows
<code>n</code>	Number of columns
<code>num</code>	Number of non-zero elements
<code>ele</code>	Pointer to dynamic array of stored elements
<code>malloc</code>	Creates dynamic memory in C
<code>free</code>	Releases dynamic memory in C
<code>&s</code>	Address of <code>s</code>
<code>struct Sparse *s</code>	Pointer to a sparse matrix
<code>-></code>	Access field through a pointer
<code>.</code>	Access field through a normal structure variable
<code>k</code>	Tracker for the current non-zero element

25. Important rules from Lesson 3

Rule 1

Use `malloc` after reading `num`.

```
s->ele = (struct Element *) malloc(s->num * sizeof(struct Element));
```

Rule 2

Use `->` inside `create()` because `s` is a pointer.

```
s->m  
s->n  
s->num  
s->ele
```

Rule 3

Use `.` inside `display()` because `s` is passed by value.

```
s.m  
s.n  
s.num  
s.ele
```

Rule 4

Use nested loops to visit every full matrix position.

```
for (i = 0; i < s.m; i++) {  
    for (j = 0; j < s.n; j++) {  
        // print value or zero  
    }  
}
```

Rule 5

Use `k` to track the current stored non-zero element.

```
int k = 0;
```

Rule 6

Print a stored value only when the current grid position matches `ele[k]`.


```
if (k < s.num && i == s.ele[k].i && j == s.ele[k].j)
```

Rule 7

If the current grid position is not stored, print zero.

```
printf("0 ");
```

26. Practice example

Given this sparse matrix information:

```
m = 3
n = 4
num = 3

ele[0] = {0, 1, 5}
ele[1] = {1, 3, 8}
ele[2] = {2, 0, 6}
```

What should `display()` print?

Answer

```
0 5 0 0
0 0 0 8
6 0 0 0
```

Explanation:

`{0, 1, 5}` means row `0`, column `1`, value `5`.

`{1, 3, 8}` means row `1`, column `3`, value `8`.

`{2, 0, 6}` means row `2`, column `0`, value `6`.

All other positions are printed as `0`.

27. Exit questions

Question 1

Why does `create()` use `struct Sparse *s`?

Answer

Because `create()` must modify the original sparse matrix from `main()`.

So we pass the address of the structure.

Question 2

Inside `create()`, should we use `s.m` or `s->m`?

Answer

Use:

`s->m`

because `s` is a pointer.

Question 3

Inside `display()`, should we use `s.m` or `s->m`?

Answer

Use:

`s.m`

because `s` is passed by value.

Question 4

What does `k` do in the display function?

Answer

`k` tracks the current non-zero element in the `ele` array.

Question 5

Why is display time $O(m * n)$?

Answer

Because display visits every visible position in the full matrix.

Question 6

Why is sparse storage space $O(\text{num})$?

Answer

Because the program stores only `num` non-zero elements.

Question 7

Why should the elements be entered in row-major order?

Answer

Because the display function moves through the `ele` array using `k` only in the forward direction.

So the stored elements must be in the same order as the display traversal.

28. Lesson 3 final summary

Lesson 3 teaches how to complete a basic C sparse matrix program.

The sparse matrix stores only non-zero elements.

Each non-zero element is stored as:

```
row, column, value
```

The `create()` function reads the matrix information and allocates memory using `malloc`.

Because `create()` receives a pointer, it uses:

```
->
```

The `display()` function prints the full matrix.

Because `display()` receives a normal structure value, it uses:

```
.
```

The display function uses nested loops to visit every position.

It uses `k` to track the current stored non-zero element.

If the current position matches `ele[k]`, it prints the stored value.

Otherwise, it prints `0`.

The main lesson is:

```
Sparse storage keeps only non-zero values, but display reconstructs the full  
matrix view by printing zeros for missing positions.
```

Complexity:

```
create() time = O(num)  
create() space = O(num)  
  
display() time = O(m * n)  
display() extra space = O(1)
```

This prepares you for the next lesson, where sparse matrices are added using a merge-style algorithm.